```
10:17:39 web.1  | * Listening on http://[::1]:3000
10:17:39 web.1  | Use Ctrl-C to stop
10:17:39 css.1  |
10:17:39 css.1  | Rebuilding...
10:17:39 css.1  | Done in 155ms.
```

Windows users will need to run the command `ruby bin/dev`.

If you examine that output, in addtion to the lines containing `web.1` that show the Rails server starting, you see lines containing `css.1` that show the CSS rebuilding. This is all controlled by a file named `Procfile.dev`:

rails7/depot_a/Procfile.dev

```
web: bin/rails server -p 3000
css: bin/rails tailwindcss:watch
```

Feel free to modify this file to suit your needs. For example, if you're using a virtual machine, you might need to add `-b 0.0.0.0` to the `rails server` line to accept connections from your host.

As with our demo application, this command starts a web server on our local host, port 3000. If you get an error saying `Address already in use` when you try to run the server, that means you already have a Rails server running on your machine. If you've been following along with the examples in the book, that might well be the Hello, World! application from Chapter 4. Find its console and kill the server using `Ctrl-C`. If you're running on Windows, you might see the prompt `Terminate batch job (Y/N)?`. If so, respond with `y`.

Let's connect to our application. Remember, the URL we give to our browser is `http://localhost:3000/products`, which has both the port number (`3000`) and the name of the controller in lowercase (`products`). The application looks like the following screenshot.



That's pretty boring. It's showing us an empty list of products. Let's add some. Click the New Product link. A form should appear, as shown in the next screenshot.